I. Basic Fuzzy Network Operations

1. Horizontal Merging

This function performs horizontal merging of two rule bases. It is called as follows:

[boolRB,inputLTCounts,outputLTCounts] = horizontal_merging_v4(boolRB1,inputLTCountsRB1,outputLTCountsRB1,boolRB2, inputLTCountsRB2,outputLTCountsRB2);

where:

- *boolRB* is the Boolean matrix representation of the resultant rule base.
- *inputLTCounts* is an array that contains the counts of the linguistic terms for each input to the resultant rule base.
- *outputLTCounts* is an array that contains the counts of the linguistic terms for each output from the resultant rule base.

- *boolRB1* is the Boolean matrix representations of the first rule base that is to be merged.
- *boolRB2* is the Boolean matrix representations of the second rule base that is to be merged.
- *inputLTCountsRB1* is an array that contains the counts of the linguistic terms for each input to the first rule base that is to be merged.
- *inputLTCountsRB2* is an array that contains the counts of the linguistic terms for each input to the second rule base that is to be merged.
- *outputLTCountsRB1* is an array that contains the counts of the linguistic terms for each output from the second rule bases that is to be merged.
- *outputLTCountsRB2* is an array that contains the counts of the linguistic terms for each output from the second rule bases that is to be merged.

Example:

```
boolRB1 = [0 1 0 0; 0 1 0 0; 0 0 1 0; 0 0 0 1];
boolRB2 = [0 0 0 0; 1 1 0 0; 0 0 0 1; 0 0 1 0];
[boolRB,inputLTCounts,outputLTCounts] = horizontal_merging_v4(boolRB1, [2
2], [2 2], boolRB2, [2 2], [2 2]);
displ_bool_rb(boolRB, inputLTCounts, outputLTCounts);
```

2. Vertical Merging

This function performs vertical merging of two rule bases. It is called as follows:

[boolRB,inputLTCounts,outputLTCounts] = vertical_merging_v4(boolRB1,inputLTCountsRB1,outputLTCountsRB1,boolRB2, inputLTCountsRB2,outputLTCountsRB2)

where:

- *boolRB* is the Boolean matrix representation of the resultant rule base.
- *inputLTCounts* is an array that contains the counts of the linguistic terms for each input to the resultant rule base.
- *outputLTCounts* is an array that contains the counts of the linguistic terms for each output from the resultant rule base.

- *boolRB1* is the Boolean matrix representations of the first rule base that is to be merged.
- *boolRB2* is the Boolean matrix representations of the second rule base that is to be merged.
- *inputLTCountsRB1* is an array that contains the counts of the linguistic terms for each input to the first rule base that is to be merged.
- *inputLTCountsRB2* is an array that contains the counts of the linguistic terms for each input to the second rule base that is to be merged.
- *outputLTCountsRB1* is an array that contains the counts of the linguistic terms for each output from the second rule bases that is to be merged.
- *outputLTCountsRB2* is an array that contains the counts of the linguistic terms for each output from the second rule bases that is to be merged.

Example:

```
boolRB1 = [0 1 0; 1 0 0; 0 0 1];
boolRB2 = [1 0 0; 0 0 1; 0 1 0];
[boolRB,inputLTCounts,outputLTCounts] = vertical_merging_v4(boolRB1, [3],
[3], boolRB2, [3], [3]);
displ_bool_rb(boolRB,inputLTCounts,outputLTCounts);
```

3. Output Merging

This function performs output merging of two rule bases. It is called as follows:

[boolRB,inputLTCounts,outputLTCounts] = output_merging_v4(boolRB1,inputLTCountsRB1,outputLTCountsRB1,boolRB2, inputLTCountsRB2,outputLTCountsRB2)

where:

- *boolRB* is the Boolean matrix representation of the resultant rule base.
- *inputLTCounts* is an array that contains the counts of the linguistic terms for each input to the resultant rule base.
- *outputLTCounts* is an array that contains the counts of the linguistic terms for each output from the resultant rule base.

- *boolRB1* is the Boolean matrix representations of the first rule base that is to be merged.
- *boolRB2* is the Boolean matrix representations of the second rule base that is to be merged.
- *inputLTCountsRB1* is an array that contains the counts of the linguistic terms for each input to the first rule base that is to be merged.
- *inputLTCountsRB2* is an array that contains the counts of the linguistic terms for each input to the second rule base that is to be merged.
- *outputLTCountsRB1* is an array that contains the counts of the linguistic terms for each output from the second rule bases that is to be merged.
- *outputLTCountsRB2* is an array that contains the counts of the linguistic terms for each output from the second rule bases that is to be merged.

Example:

```
boolRB1=[1 1; 1 0; 0 1; 0 0];
boolRB2=[0 1; 1 0; 1 0; 0 0];
[boolRB,inputLTCounts,outputLTCounts] = output_merging_v4(boolRB1, [2 2],
[2], boolRB2, [2 2], [2]);
displ_bool_rb(boolRB,inputLTCounts,outputLTCounts);
```

II. Advanced Fuzzy Network Operations

1. Input Augmentation

This function performs input augmentation for a rule base. It is called as follows:

[productBoolRB, productInputLTCounts, productOutputLTCounts] = input_augmentation(boolRB, inputLTCounts, outputLTCounts, augmentationPosition, augmentedInputLTCount)

where:

- *productBoolRB* is the Boolean matrix representation of the resultant rule base.
- *productInputLTCounts* is an array that contains the counts of the linguistic terms for each input to the resultant rule base.
- *productOutputLTCounts* is an array that contains the counts of the linguistic terms for each output from the resultant rule base.

- *boolRB* is the Boolean matrix representations of the rule base that is to be augmented.
- *inputLTCounts* is an array that contains the counts of the linguistic terms for each input to the rule base that is to be augmented.
- *outputLTCounts* is an array that contain the counts of the linguistic terms for each output from the rule base that is to be augmented.
- *augmentationPosition* is the position of the augmented input in the rule base that is to be augmented.
- *augmentedInputLTCount* is the number of linguistic terms for the augmented input.

Example:

```
N = [0 1 0; 0 0 1; 1 0 0];
[prodRB, inLT, outLT] = input_augmentation(N,[3],[3],1,3);
displ_bool_rb(prodRB, inLT, outLT);
```

2. Output Permutation

This function performs output permutation for a rule base. It is called as follows:

[productBoolRB, productInputLTCounts, productOutputLTCounts] = output_permutation(boolRB, inputLTCounts, outputLTCounts, newOutputPermutation)

where:

- *productBoolRB* is the Boolean matrix representation of the resultant rule base.
- *productInputLTCounts* is an array that contains the counts of the linguistic terms for each input to the resultant rule base.
- *productOutputLTCounts* is an array that contains the counts of the linguistic terms for each output from the resultant rule base.

- *boolRB* is the Boolean matrix representations of the rule base to be permuted.
- *inputLTCounts* is an array that contains the counts of the linguistic terms for each input to the rule base that is to be permuted.
- *outputLTCounts* is an array that contains the counts of the linguistic terms for each output from the rule base that is to be permuted.
- *newOutputPermutation* is an array that contains the output permutation to be applied.

Example:

```
N = [0 0 0 0 0 1 0 0 0; 0 0 0 0 0 0 1 0 0; 0 1 0 0 0 0 0 0 0];
[prodRB, inLT, outLT] = output_permutation(N,[3],[3 3],[2 1]);
displ_bool_rb(prodRB, inLT, outLT);
```

3. Feedback Equivalence

This function performs feedback equivalence for a rule base. It is called as follows:

[productBoolRB, productInputLTCounts, productOutputLTCounts] = feedback_equivalence(inputLTCounts, outputLTCounts, feedbackIndexes)

where:

- *productBoolRB* is the Boolean matrix representation of the resultant rule base.
- *productInputLTCounts* is an array that contains the counts of the linguistic terms for each input to the resultant rule base.
- *productOutputLTCounts* is an array that contains the counts of the linguistic terms for each output from the resultant rule base.

- *inputLTCounts* is an array that contains the counts of the linguistic terms for each input to the rule base that is to be equivalenced.
- *outputLTCounts* is an array that contains the counts of the linguistic terms for each output from the rule base that is to be equivalenced.
- *feedbackIndexes* is an array with two columns, such that each row of this array [Yi,Xi] represents an existing feedback connection from an output with position Yi to an input with position Xi within the rule base that is to be equialenced.

Example:

```
[prodRB, inLT, outLT] = feedback_equivalence([2 2],[2 2],[1 1]);
displ_bool_rb(prodRB, inLT, outLT);
```